

NumPy 초보자 종합 입문 가이드

파이썬 리스트의 한계부터 배열 생성, 구조 파악, 형태 변환까지 핵심 개념 완벽 정리

1. NumPy란 무엇인가? (개념과 탄생 배경)

NumPy(Numerical Python)는 파이썬에서 대규모 수치 데이터와 행렬 계산을 빠르고 효율적으로 처리하기 위해 만들어진 **핵심 라이브러리**입니다. 공장 설비 데이터(온도, 진동, 압력 센서 등)처럼 1초에도 수천~수만 개씩 쌓이는 데이터 묶음을 다룰 때 파이썬 기본 기능만으로는 한계가 있습니다. NumPy는 이러한 수백만 개의 숫자 데이터를 한 번에 빠르게 계산할 수 있는 최적의 도구를 제공합니다.

💡 NumPy는 데이터 분석 스택의 뿌리(Foundation)

데이터 분석에서 자주 사용하는 **Pandas**(표 데이터 처리), **Matplotlib / Seaborn**(데이터 시각화), **Scikit-learn / PyTorch**(머신러닝 및 AI) 등의 모든 고급 도구들이 내부적으로는 **NumPy 배열(ndarray)** 구조 위에서 작동합니다. 따라서 NumPy를 이해하는 것이 데이터 분석 및 AI 학습의 가장 첫걸음입니다.

2. 파이썬 리스트 vs NumPy 배열 (왜 NumPy를 써야 하는가?)

파이썬 기본 구조인 **List**와 NumPy의 핵심 구조인 **Array (ndarray)**는 비슷해 보이지만 internal 처리 방식과 성능에서 결정적인 차이가 있습니다.

① 연산 방식의 차이: 반복문 vs vectorize (묶음 연산)

파이썬 리스트에 숫자를 곱하면 값이 계산되는 것이 아니라 **리스트가 반복(복사)**됩니다. 리스트의 모든 요소에 숫자를 더하거나 곱하려면 **for** 문(반복문)을 사용해야 합니다. 반면 NumPy 배열은 반복문 없이 한 줄로 묶음 연산(Vectorized Operation)을 수행합니다.

```
# [파이썬 리스트] - 반복문 필요
py_list = [70.5, 69.8, 73.7]
result = [x + 5 for x in py_list] # [75.5, 74.8, 78.7]

# [NumPy 배열] - 한 줄로 전체 계산 (Vectorization)
import numpy as np
np_arr = np.array([70.5, 69.8, 73.7])
print(np_arr + 5) # 출력: [75.5 74.8 78.7]
```

② 속도의 비결: 동적 타입 vs 단일 자료형 (Homogeneous Type)

파이썬 리스트는 정수, 실수, 문자열 등 **다양한 자료형**을 섞어 담을 수 있지만, 이 때문에 메모리에 데이터가 파편화되어 저장되고 속도가 느립니다. 반면 NumPy 배열은 **오직 같은 자료형(Single Data Type)**만 담을 수 있습니다. 데이터가 메모리에 연쇄적으로(Contiguous Memory) 정렬되어 C언어 수준의 빠른 속도로 연산이 가능합니다.

비교 항목	파이썬 리스트 (Python List)	NumPy 배열 (ndarray)
자료형 (Data Type)	서로 다른 자료형 혼용 가능 (글자, 숫자 등)	동일한 자료형만 저장 가능 (통일됨)
전체 연산 처리	<code>for</code> 반복문이나 리스트 내포 작성 필요	반복문 없이 한 줄로 묶음 연산 가능
처리 속도 및 메모리	대용량 데이터 처리 시 상대적으로 느림	C 언어 기반 연속 메모리 배치로 매우 빠름
출력 형태	쉼표로 구분: <code>[1, 2, 3]</code>	공백으로 구분: <code>[1 2 3]</code>

3. NumPy 불러오기 규칙

NumPy를 사용할 때는 관례적으로 `np` 라는 별칭(alias)을 붙여 불러옵니다. 이는 전 세계 모든 데이터 분석가가 공유하는 공통 약속입니다.

```
# NumPy 라이브러리 불러오기
import numpy as np

# 데이터 분석 3대 라이브러리 불러오기 약속
import numpy as np          # 숫자 묶음 및 행렬 연산
import pandas as pd         # 표 데이터 처리
import matplotlib.pyplot as plt # 그래프 시각화
```

4. 다양한 배열 생성 방법

1) np.array() - 기존 값으로 생성

리스트나 튜플을 받아 배열로 변환합니다. 하나라도 소수점이 있으면 전체가 실수형(float)으로 자동 통일됩니다.

```
# 1차원 배열
a = np.array([1, 2, 3])

# 자동 형변환 (실수 통일)
b = np.array([1, 2, 3.0])
# [1. 2. 3.]
```

2) np.arange() - 간격 중심 연속값

시작, 끝, 간격을 지정해 배열을 만듭니다. '시작 포함, 끝 제외' 파이썬 표준 규칙을 따릅니다.

```
# 0부터 4까지 생성 (5 제외)
np.arange(5) # [0 1 2 3 4]

# 0부터 8까지 2 간격
np.arange(0, 10, 2) # [0 2 4 6 8]
```

3) np.linspace() - 개수 중심 균등 분할

시작과 끝 구간을 지정한 개수만큼 정확히 나눕니다. 끝값을 포함한다는 점이 `arange` 와 다릅니다.

```
# 0부터 1까지 5개로 균등 분할
np.linspace(0, 1, 5)
# [0. 0.25 0.5 0.75 1.]
```

4) np.zeros() / ones() / full()

특정 값으로 채워진 초기화 배열을 생성합니다. 연산 결과를 담을 빈 그릇이나 가중치 초기화 시 활용됩니다.

```
# 0으로 채우기 / 특정값 채우기
np.zeros(3) # [0. 0. 0.]
np.full(3, 7) # [7 7 7]
```

🔍 함수 선택 가이드: 무엇을 써야 할까?

- 이미 측정된 데이터 값이 있는가? → `np.array()`

- 측정 시간축처럼 '일정한 간격'이 중요한가? → `np.arange()`
- 그래프 X축처럼 구간을 '정해진 개수'로 똑같이 나눌 것인가? → `np.linspace()`
- 계산 결과를 담을 빈 배열(0이나 1)을 미리 만들어 둘 것인가? → `np.zeros()` / `np.ones()`

5. 배열의 구조와 4대 필수 속성 (Attributes)

NumPy 배열을 만들거나 불러온 후에는 가장 먼저 데이터의 구조를 파악해야 합니다. 속성은 이미 계산되어 저장된 정보이므로 괄호 `()` 를 붙이지 않습니다. (예: `arr.shape` (O), `arr.shape()` (X))

속성명 (Attribute)	의미 및 설명	코드 및 예시 출력
<code>ndim</code>	차원의 개수 (Number of Dimensions) 1차원(선)=1, 2차원(표)=2	<code>arr.ndim</code> → 2
<code>shape</code>	배열의 형태 (행, 열 크기) 튜플 (행, 열) 형태로 반환	<code>arr.shape</code> → (2, 3) (2행 3열)
<code>size</code>	전체 원소(값)의 총 개수 행 크기 × 열 크기와 동일	<code>arr.size</code> → 6
<code>dtype</code>	원소의 자료형 <code>int64</code> (정수), <code>float64</code> (실수) 등	<code>arr.dtype</code> → <code>float64</code>

⚠ `astype()` 사용 시 주의사항 (자료형 변환)

`arr.astype(int)` 를 사용해 실수를 정수로 변환할 때, 반올림이 아니라 소수점 아래를 버립니다. (예: 1.9 → 1)
또한 `astype()` 은 원본을 직접 바꾸지 않고 새로운 배열을 반환하므로, 반드시 `new_arr = arr.astype(int)` 처럼 변수에 새로 저장해야 합니다.

6. 배열의 형태 변환 (reshape & flatten)

데이터 분석 시 1차원 데이터로 들어온 센서 측정값들을 (시점, 센서) 형태의 2차원 표로 재배치하거나, 반대로 2차원 표를 한 줄로 펼치는 작업이 빈번하게 일어납니다.

① `reshape()`: 형태 바꾸기

원소의 총 개수(`size`)를 유지하면서 원하는 행과 열 모양으로 재배치합니다. 변환 전후의 총 개수가 맞지 않으면 오류가 발생합니다.

```
# 0~11까지 12개 원소를 가진 1차원 배열
a = np.arange(12)

# 3행 4열 2차원 배열로 변환
b = a.reshape(3, 4)

# -1 활용: 열을 2개로 고정하고 행은 자동 계산 (12 / 2 = 6행)
c = a.reshape(-1, 2)
```

② flatten(): 1차원으로 펴기

2차원 표 형태의 배열을 책 읽는 방향(첫 번째 행 왼쪽→오른쪽, 두 번째 행 왼쪽→오른쪽...)으로 읽어 한 줄짜리 1차원 배열로 펼칩니다.

```
# 2행 3열 표 생성
b = np.array([[1, 2, 3], [4, 5, 6]])

# 1차원 배열로 펼치기
print(b.flatten()) # 출력: [1 2 3 4 5 6]
```